

IF 45 I

Advance Web Programming

Meet II - Connecting Express API to ReactJS

Wirawan Istiono, S.Kom., M.Kom



Meet I I

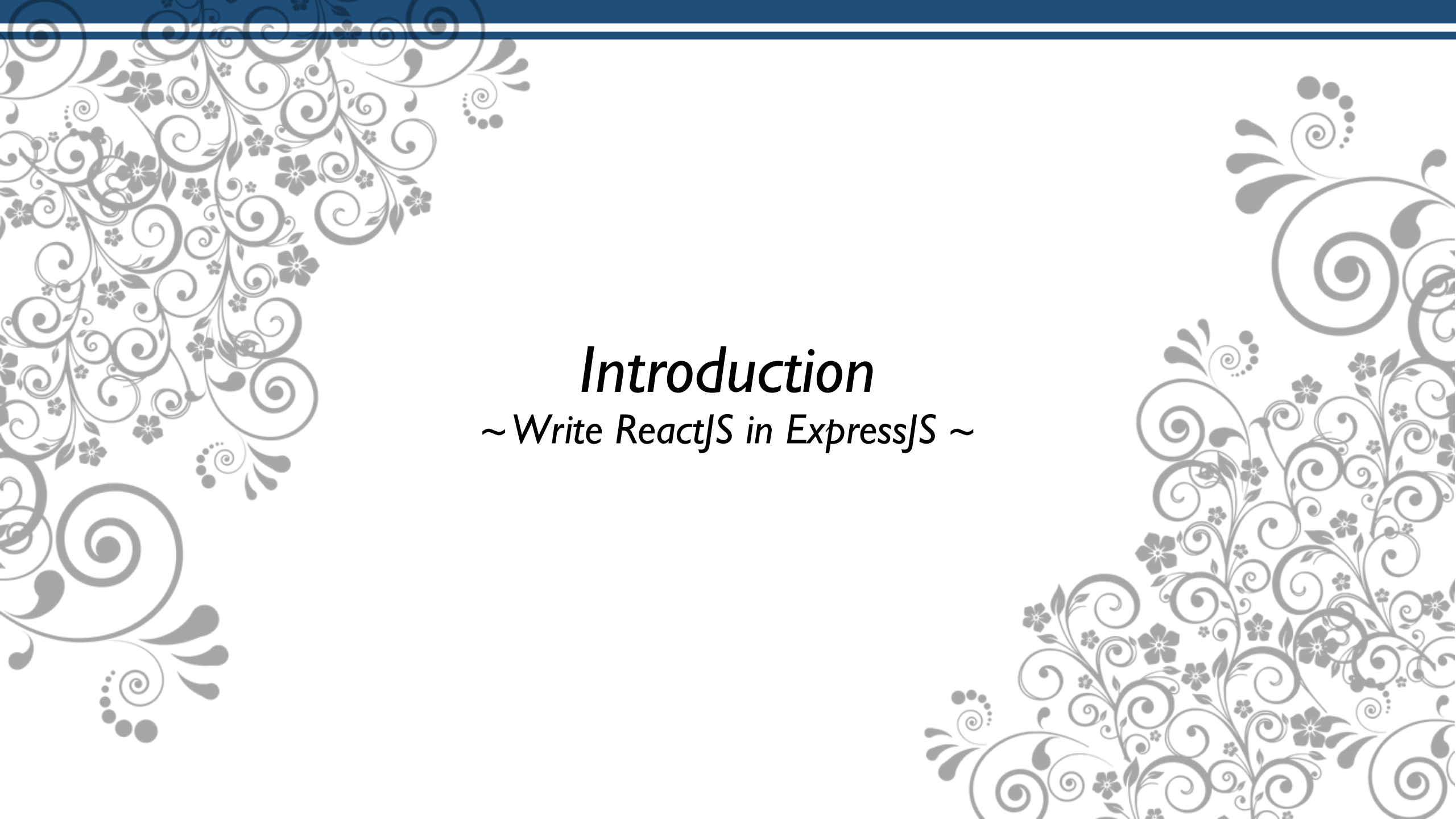
Objective:

Sub-CLO07I3 - Students are able to implement various types of libraries and database to create web application using Node JS (C3)

Sub-CLO07I4 - Students are able to implement Node JS Framework to build Web application (C3, C6)

OUTLINE

- *CORS handling.*
- *Fetch API in ReactJS*

The slide features a white background with decorative gray floral and scrollwork patterns in the top-left and bottom-right corners. A solid dark blue horizontal bar is positioned at the very top of the slide.

Introduction

~ Write ReactJS in ExpressJS ~

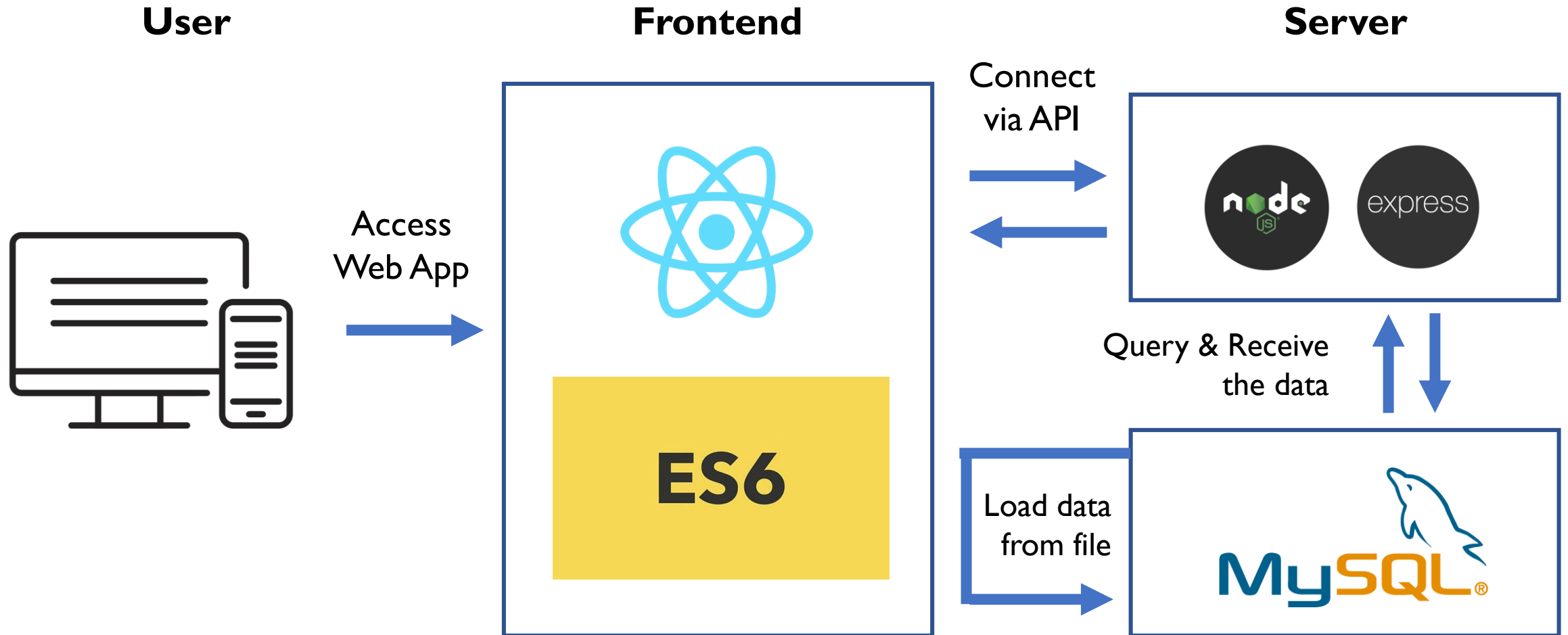
Rules to write ReactJS in Express or Node

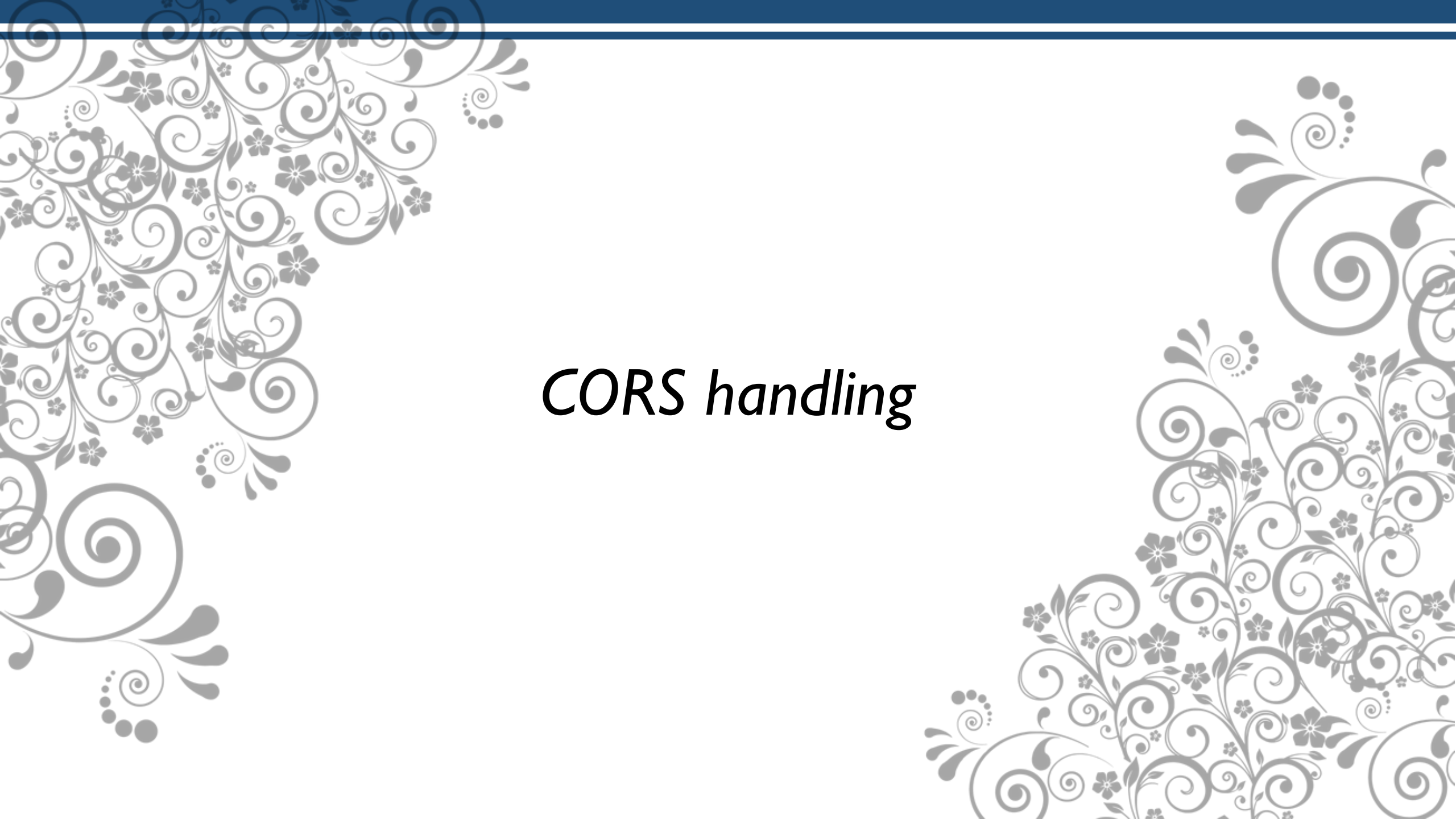
- You cannot directly write ReactJS (JSX) inside ExpressJS like normal HTML
- ReactJS must be built separately (using tools like Vite or Create React App), then the built static files (HTML, JS, CSS) are served by Express.

Why?

- ReactJS uses JSX syntax (e.g. `<div>{variable}</div>`), which browsers don't understand directly.
- JSX must be compiled (transpiled by Babel/Vite/Webpack) into plain JavaScript before it can run in the browser.
- ExpressJS is a backend framework — it serves APIs or static files, not JSX code.

Architecture ReactJS and NodeJS (ExpressJS)





CORS handling

What is CORS Handling?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that prevents a frontend app (for example ReactJS running on `http://localhost:3001`) from accessing an API server (for example ExpressJS running on `http://localhost:3000`) unless the backend explicitly allows it.

Without enabling CORS, React requests to Express will fail with an error:

Access to fetch at 'http://localhost:3000/api' from origin 'http://localhost:3001' has been blocked by CORS policy.

CORS - How to Serve React with Express

There are two common approaches:

1. Separate Frontend & Backend (most common)
 - Run React on one server (e.g., `http://localhost:3001`).
 - Run Express API on another server (e.g., `http://localhost:3000`).
 - React calls Express using `fetch` or `axios`.

2. Serve React Build from Express
 - Build React project (`npm run build`).
 - Copy the `build/` (CRA) or `dist/` (Vite) folder into Express.
 - Use `express.static` to serve it.

Mini Project (ReactJS and ExpressJS)

Step I - Create project structure

Create folder and files like the sample below

Your-projctet-app/

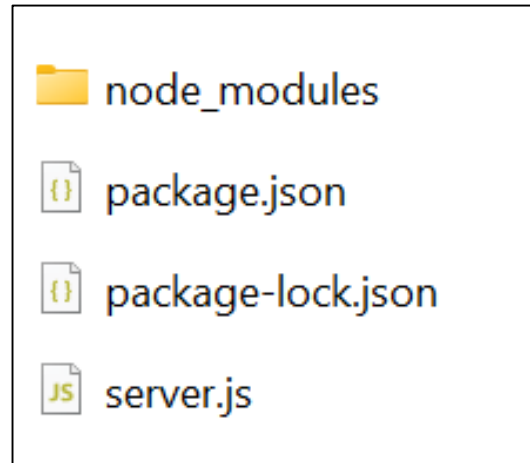
```
|— backend/  
|   |— server.js  
|— frontend/  
|   |— (React project files)
```

Mini Project (ReactJS and ExpressJS)

Step2 – Setup Express (Backend)

Inside the **backend** folder, Initialize Node project using this command:

```
cd backend  
npm init -y  
npm install express cors
```



Mini Project (ReactJS and ExpressJS)

Step3 – Setup ReactJS - Vite (Frontend)

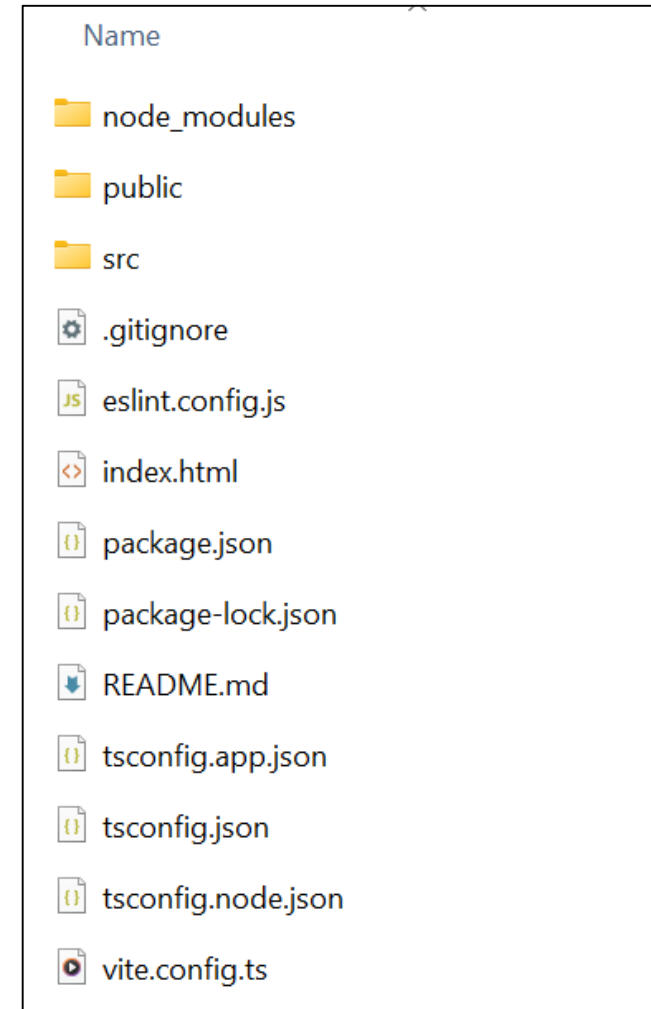
Inside the **root and frontend** folder, to create React project with Vite using this command:

In project root folder:

- `npm create vite@latest frontend`
- Choose React and typescript
- Follow the step with default options

In frontend folder:

- `npm install`
- `npm install axios`



Mini Project (ReactJS and ExpressJS)

Step4 – Edit server.js

Open your server.js
file in the folder
backend, and write
the script beside.

```
const express = require("express");
const path = require("path");
const cors = require("cors");
const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// API routes
app.get("/api/hello", (req, res) => {
  res.json({ message: "Hello from Express backend!" });
});
app.post("/api/echo", (req, res) => {
  const { name } = req.body;
  res.json({ reply: `Hi ${name}, your data was received!` });
});

// Serve frontend (production)
app.use(express.static(path.join(__dirname, "../frontend/dist")));
app.use((req, res) => {
  res.sendFile(path.join(__dirname, "../frontend/dist/index.html"));
});

app.listen(3000, () =>
  console.log(`Server running on http://localhost:3000`)
);
```

Mini Project (ReactJS and ExpressJS)

Step5 – Edit app.tsx

In your **frontend** folder, open **src/App.tsx** and edit the code like the sample below.

```
import { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [message, setMessage] = useState("");
  const [name, setName] = useState("");
  const [reply, setReply] = useState("");

  // GET request to backend
  useEffect(() => {
    axios.get("http://localhost:3000/api/hello")
      .then(res => setMessage(res.data.message))
      .catch(err => console.error(err));
  }, []);
```

```
// POST request to backend
const sendData = async () => {
  try {
    const res = await
axios.post("http://localhost:3000/api/echo", { name });
    setReply(res.data.reply);
  } catch (err) {
    console.error(err);
  }
};

return (
  <div>
    <p>{message}</p>

    <input type="text" placeholder="Enter your name"
value={name}
    onChange={(e) => setName(e.target.value)} />
    <input type='button' onClick={sendData} value='kirim' />

    {reply} && <p>Backend reply: {reply}</p>
  </div>
);
}

export default App;
```

Mini Project (ReactJS and ExpressJS)

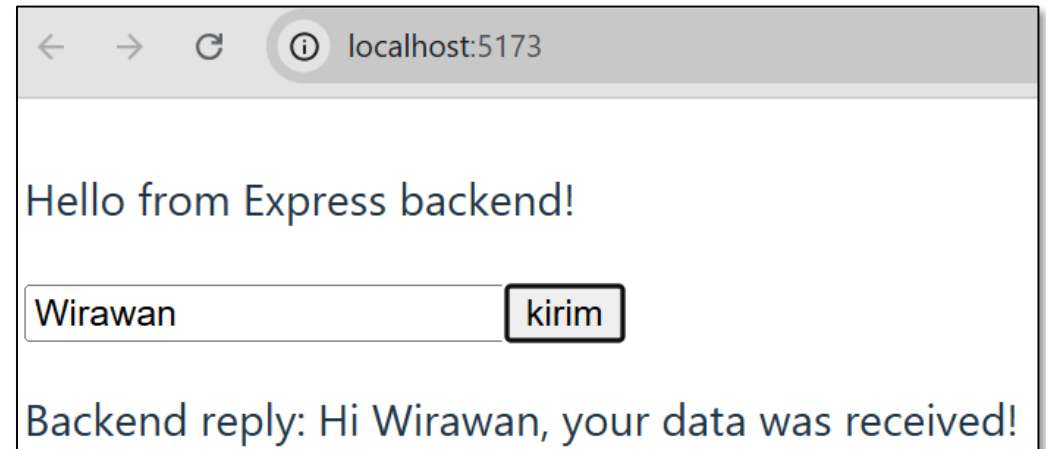
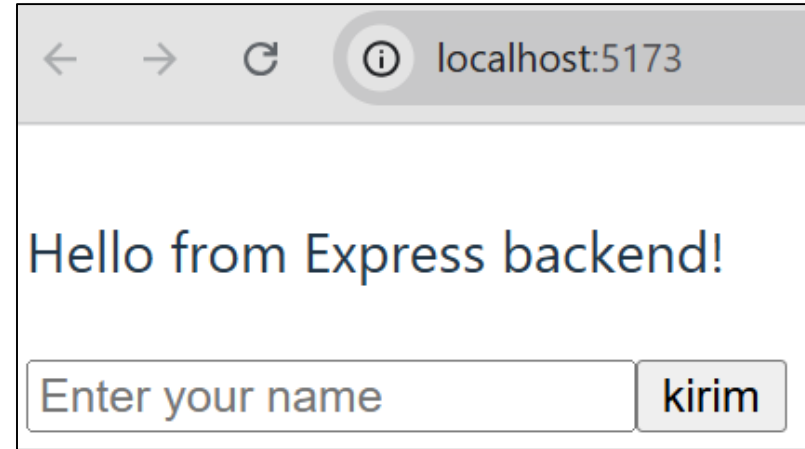
Step5 – Run Development Servers

Run backend in the command prompt, and write the command below to start server

```
cd backend  
node server.js
```

Run frontend In the new command prompt, and write the command below to start frontend

```
cd frontend  
npm run dev
```





Fetch API in ReactJS

What is Fetch API in ReactJS?

The Fetch API is a built-in JavaScript feature used to make HTTP requests (GET, POST, PUT, DELETE) from the frontend (React) to the backend (Express API).

React can call the Express API using `fetch()` (or other libraries like `axios`).

Example ReactJS with Fetch API

Suppose React runs at

`http://localhost:5173`

and

Express API at

`http://localhost:3000`

```
import React, { useEffect, useState } from "react";

function App() {
  const [data, setData] = useState("");

  useEffect(() => {
    fetch("http://localhost:3000/api/data") // Call Express API
      .then(response => response.json())
      .then(result => setData(result.message))
      .catch(error => console.error("Error:", error));
  }, []);

  return (
    <div>
      <h1>React + Express Example</h1>
      <p>Message from API: {data}</p>
    </div>
  );
}

export default App;
```

REFERENCES

- B. Lawson and J. Encinas, Node.js Web Development, 6th ed., Packt Publishing, 2023
- R. Raj, Learning Node.js Development, Packt Publishing, 2018.
- E. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, Node.js in Action, 2nd ed., Manning Publications, 2017.
- Maulana, a., bau, r.t.r., munawar, z., setiawan, r., aisa, s., istiono, w., irfan, a., pratama, y.a., ramadhany, e.d., pomalingo, s. And permana, a.a., 2023. *Pemrograman web 101: memahami dasar-dasar untuk mengembangkan situs web*. Get press indonesia.
- The Node.js Foundation, “Node.js Documentation,” [Online]. Available: <https://nodejs.org/en/docs/>. [Accessed: Jun. 5, 2025].
- Express.js Team, “Express.js Documentation,” [Online]. Available: <https://expressjs.com/>. [Accessed: Jun. 5, 2025].

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.



Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.